Project Report: Placement Portal Application

Course: Modern Application Development - II (MAD - II)

1. Student Details

- **Name:** Ansh Garg
- **Student ID:** 24f1001800@ds.study.iitm.ac.in

2. Project Details

- **Problem Statement:** The project was to evolve the campus recruitment Placement Portal into a modern, production-grade distributed application. While maintaining the core interaction between the Admin, Companies, and Students, the system must now process decoupled client-side rendering, handle heavy file exports asynchronously via a distributed background worker pool, and implement automated time-based periodic tasks.
- **My Approach:** I decoupled the entire application by splitting it into a stateless backend API and a responsive frontend client, integrating a distributed background architecture for optimization:
    - **Frontend (Progressive PWA):** Built as a Single Page Application (SPA) using Vue.js. It manages local application state dynamically and communicates with the backend solely via asynchronous JSON requests using Axios interceptors to inject JWT security contexts.
    - **Backend REST API (app.py & Blueprints):** Transformed into a stateless API layer. Routing is modularized into distinct file layers (auth_api.py, admin_api.py, company_api.py, student_api.py) registered via Flask Blueprints under an explicit /api namespace prefix.
    - **Distributed Background Workers (tasks.py):** Configured a task broker pipeline using Redis and Celery. Heavy file inputs/outputs (generating applicant lists for companies and historical logs for students) are offloaded to isolated worker processes to keep the core server responsive.
    - **Automated Clock Scheduler:** Integrated Celery Beat to automatically execute time-critical background tasks (daily recruitment deadlines and monthly analytical metrics) without human intervention.

3. Frameworks and Libraries Used

- **Vue.js & Tailwind CSS:** For reactive client-side rendering, progressive routing, and dynamic data-binding.
- **Axios:** For handling asynchronous AJAX HTTP communication.

- **Flask & Flask-JWT-Extended:** Micro-framework used to serve stateless endpoints protected by cryptographically signed token payloads.
- **Flask-CORS:** To manage secure Cross-Origin Resource Sharing boundaries between the client and server.
- **Flask-SQLAlchemy & SQLite:** Database ORM framework mapping relational tables.
- **Redis Engine:** Deployed as the in-memory message broker bus for distributing background execution queues.
- **Celery & Celery Beat:** The background task processing architecture and periodic crontab timing system.

4. AI/LLM Declaration

I declare that I used an AI (Gemini) as a technical collaborator. I used it for:

- **Decoupling State Logic:** Structuring the asynchronous frontend polling framework (setInterval) to cleanly request Celery task completion statuses.
- **Defensive Backend Query Engineering:** Implementing case-insensitive query structures (.ilike()) and token fallback conditions to resolve synchronization issues during state changes.
- **Task Architecture Integration:** Configuring Celery application context wrappers to safely read database rows outside the core web runtime thread.

5. Relational Schema Layout (Database Structure)

1. **User Table:** id, email, password, role, company_name *(Unified schema optimization)*.
2. **StudentProfile Table:** id, user_id, name, branch, cgpa, year, resume_url.
3. **PlacementDrive Table:** id, company_id, job_title, description, cgpa_eligibility, branch_eligibility, deadline, status *(Pending/Approved/Closed)*.
4. **Application Table:** id, student_id, drive_id, status *(Applied/Shortlisted/Selected/Rejected)*, interview_date.

6. Distributed Tasks & Background Architectures

Asynchronous Compilation Tasks (Requirement C)

Expensive operations like building and downloading CSV files are offloaded from the server's request-response loop:

- **Trigger Endpoint:** POST /api/student/export-history (and /api/company/export-applicants).

- **Execution Flow:** Flask sends the function to Redis and yields an instant 202 Accepted status back to the user interface along with a tracking ID.
- **Worker Processing:** A background Celery worker isolates the query, builds the file in the /exports folder, and caches the download URL. The frontend regularly checks the status via /export-status/<task_id> and transforms the view into a green download button upon success.

Automated Background Batch Scheduler (Requirements A & B)

Using Celery Beat crontab mapping configurations, the ecosystem runs timekeeper loops independent of user dashboard interactions:

- **Daily Reminders (Requirement A):** Automatically executes every midnight (crontab(hour=0, minute=0)) to scan database rows for application windows closing within 24 hours, printing system-wide notification hooks.
- **Monthly Activity Reports (Requirement B):** Automatically runs on the 1st day of every month at midnight (crontab(day_of_month=1, hour=0, minute=0)) to process cumulative recruitment stats, generating structural analytics for the Admin logs.

7. Decoupled REST API Resource Endpoints

The platform runs entirely as a stateless API surface using JSON payloads for data transfer:

- POST /api/auth/register & POST /api/auth/login (Returns secure JWT tokens)
- GET /api/admin/dashboard & POST /api/admin/drives/approve/<id>
- GET /api/company/applicants & POST /api/company/export-applicants
- GET /api/student/profile & POST /api/student/profile
- POST /api/student/apply & POST /api/student/export-history

8. Architectural Evolution: MAD-1 vs. MAD-2

Architectural	MAD-1 Architecture (Monolith)	MAD-2 Layout (Decoupled & Distributed)
Presentation	**Monolithic & Coupled:** Server-side layout generation using Jinja2 templates and Bootstrap	**Decoupled PWA client:** Client-side progressive web application using Vue.js for fluid rendering
Task Concurrency	**Synchronous Blocking:** Generating downloads halts the primary server instance, blocking	**Asynchronous Background Processing:** Requests are offloaded via Redis to detached
Automation	**Manual Operations:** Cron processes or date updates require an administrative user to click a	**Autonomous Beat Loops:** Automated tasks run on schedule via background

Session Tracki	**Stateful Server Cookies:** Relying on session storage limits horizontal system scaling	**Stateless Bearer Tokens:** Encrypted JWT keys are saved directly in the client's browser

9. Presentation Video Link

https://drive.google.com/file/d/1g7qgBgem6hlBCvf5Hm47JDGLMKhWFFYs/view?usp=drive_link